Universidad de Caldas
Bases de datos no relacionales
Plantilla Entrega 1. Proyecto Final

Esta plantilla corresponde a la Entrega 1: Diseño y Configuración del Sistema RAG NoSQL con MongoDB. Reemplacen todo texto entre corchetes `[ ]`, conserven títulos y numeración, y no eliminen la sección final de evaluación (la diligencia el docente).

- Entreguen en PDF con el nombre `E1_NoSQL_[Apellido1]_[Apellido2]_[Apellido3].pdf`.
- Los documentos de ejemplo deben ser JSON válido (pueden usar notación extendida de MongoDB: `ObjectId(...)`, `ISODate(...)`).
- Todos los scripts (inicialización, validación, índices) deben estar en el repositorio del grupo.
- Grupos de 2 o 3 integrantes.

Campo	Información del grupo
Nombre del proyecto	[ ]
Dominio de datos	[ej. noticias científicas con fotografías]
Modalidad	[Pareja / Terna]
Integrante 1 (nombre · código · correo)	[ ]
Integrante 2 (nombre · código · correo)	[ ]
Integrante 3 (nombre · código · correo)	[ ]
Repositorio Git	[URL]
Infraestructura	[Opción A: Atlas M0 / Opción B: MongoDB local en Docker]
Lenguaje de la API	[Python + FastAPI / Node.js + Express]
Fecha de entrega	[ ]

1. Documento de análisis

1.1 Universo del discurso

[En 1 a 2 párrafos: qué contexto modela el sistema, qué preguntas debe responder el RAG y quién lo usa. Indiquen por qué los datos son heterogéneos o

semiestructurados y por qué un modelo de documentos es adecuado. Por que se requiere una base de datos no relacional para este proyecto]

1.2 Fuentes y volumen de datos

Para la Entrega 2 se exigen al menos 100 documentos de texto y 50 imágenes.

1.3 Requerimientos funcionales

ID	Requerimiento	Operación MongoDB que lo satisface	Endpoint
RF-01	[Búsqueda semántica sobre el contenido]	$vectorSearch	POST /search
RF-02	[Filtrar por idioma y fecha con similitud]	$vectorSearch con filter / compound	POST /search
RF-03	[Búsqueda de texto completo]	Índice de texto / $search text	POST /search
RF-04	[Imágenes similares a una imagen o a un texto]	$vectorSearch sobre embeddings CLIP	POST /search
RF-05	[Respuesta generada con contexto]	Recuperación + LLM	POST /rag
RF-06	[Historial de consultas de un usuario]	[Lectura del arreglo embebido]	[ ]
RF-0n	[ ]	[ ]	[ ]

1.4 Patrones de acceso

En NoSQL el esquema se diseña a partir de las consultas. Listen las más frecuentes.

#	Patrón de acceso	Frecuencia (alta / media / baja)	Lectura o escritura	Colecciones involucradas
1	[Recuperar top-k chunks por similitud con filtro de idioma]	[ ]	[ ]	[ ]
2	[ ]	[ ]	[ ]	[ ]
3	[ ]	[ ]	[ ]	[ ]

2. Justificación del modelado NoSQL

2.1 Embedding vs. referencing por relación

Una fila por cada relación entre entidades del dominio. Cada decisión debe apoyarse en un patrón de acceso de la sección 1.4.

Relación	Cardinalidad	Estrategia (embedded / referenced / híbrida)	Patrón de acceso que la justifica	Riesgo considerado (tamaño, duplicación, crecimiento)
Usuario → historial de consultas	1:N acotado	Embedded	[ ]	[ej. limitar a las últimas N consultas]
Documento → metadatos	1:1	Embedded	[ ]	[ ]
Documento → imágenes	1:N	Referenced / híbrida	[ ]	[binarios grandes, límite 16 MB por documento]
Documento → chunks	1:N	Referenced	[ ]	[cientos de chunks con vectores de 384 dimensiones]
Documento ↔ usuarios que lo comparten	N:M	Referenced	[ ]	[ ]
[ ]	[ ]	[ ]	[ ]	[ ]

2.2 Comparación general

Criterio	Embedding	Referencing	Decisión del grupo
Lecturas en una sola operación	[ ]	[ ]	[ ]
Actualizaciones y consistencia	[ ]	[ ]	[ ]
Tamaño y crecimiento del documento	[ ]	[ ]	[ ]

Criterio	Embedding	Referencing	Decisión del grupo
Datos compartidos entre documentos	[ ]	[ ]	[ ]
Necesidad de $lookup	[ ]	[ ]	[ ]

3. Diseño del esquema NoSQL

3.1 Catálogo de colecciones

Colección	Propósito	Estrategia (embedded / referenced / híbrida)	Referencias a otras colecciones	Volumen estimado
usuarios	[ ]	[Embedded: historial de consultas]	[ ]	[ ]
documentos	[ ]	[Híbrida: metadatos embebidos, imágenes referenciadas]	imagenes	[≥ 100]
imagenes	[ ]	[Referenced]	documentos	[≥ 50]
chunks	Fragmentos vectorizados de texto	Referenced	documentos	[ ]
consultas	[ ]	[ ]	usuarios	[ ]
[evaluaciones, opcional para RAGAS]	[ ]	[ ]	consultas	[ ]
[ ]	[ ]	[ ]	[ ]	[ ]

3.2 Documentos de ejemplo

Incluyan un documento de ejemplo completo y realista por colección, con todos los campos.

documentos

```
{
  "_id": "ObjectId(...)",
  "titulo": "[ ]",
  "idioma": "es",
  "fecha": "ISODate(...)",
  "fuente": "[ ]",
  "contenido_texto": "[ ]",
  "metadatos": { "autor": "[ ]", "categoria": "[ ]", "etiquetas": ["[ ]", "[ ]"]
},
  "imagenes": ["ObjectId(...)", "ObjectId(...)"]
}
```

chunks (obligatorio: campo `estrategia_chunking`)

```
{
  "_id": "ObjectId(...)",
  "doc_id": "ObjectId(ref al documento padre)",
  "chunk_index": 3,
  "estrategia_chunking": "sentence-aware",
  "chunk_texto": "El contenido del fragmento...",
  "embedding": [0.023, -0.117, "... 384 valores"],
  "modelo": "all-MiniLM-L6-v2",
  "idioma": "es",
  "fecha": "ISODate(...)",
  "fecha_ingesta": "ISODate(...)"
}
```

imagenes

```
{
  "_id": "ObjectId(...)",
  "doc_id": "ObjectId(...)",
  "ruta_archivo": "[ruta o URL]",
  "descripcion": "[ ]",
  "etiquetas": ["[ ]"],
  "embedding": ["... 512 valores CLIP"],
  "modelo": "clip-vit-base-patch32"
}
```

Importante: el embedding de imagen se calcula sobre el contenido visual del archivo con CLIP, no sobre la cadena `ruta_archivo`.

usuarios, **consultas** y demás colecciones: [ejemplos aquí]

4. Índices y schema validation

4.1 Estrategia de índices

Colección	Índice	Tipo	Campos	Consulta o patrón que lo usa
documentos	idx_fecha_idioma	Compuesto (obligatorio)	{ fecha: 1, idioma: 1 }	[ ]

Colección	Índice	Tipo	Campos	Consulta o patrón que lo usa
documentos	idx_texto	Texto (obligatorio)	{ contenido_texto: "text" }	[ ]
chunks	vector_index	Vector Search (Atlas) o equivalente local	embedding (384, cosine) + filtros	[ ]
imagenes	vector_index_img	Vector Search	embedding (512, cosine)	[ ]
chunks	[ ]	[Simple / compuesto]	{ doc_id: 1, estrategia_chunking: 1 }	[ ]
[ ]	[ ]	[ ]	[ ]	[ ]

Definición de referencia del índice vectorial (Atlas):

```
{
  "fields": [
    { "type": "vector", "path": "embedding", "numDimensions": 384, "similarity":
"cosine" },
    { "type": "filter", "path": "estrategia_chunking" },
    { "type": "filter", "path": "idioma" }
  ]
}
```

Si usan la opción local, expliquen cómo resolverán la búsqueda vectorial: [ ]

4.2 Reglas de schema validation

Incluyan el `$jsonSchema` de al menos las colecciones documentos y chunks.

```
db.createCollection("chunks", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["doc_id", "chunk_index", "estrategia_chunking", "chunk_texto",
"embedding", "modelo"],
    properties: {
      doc_id: { bsonType: "objectId" },
      chunk_index: { bsonType: "int", minimum: 0 },
      estrategia_chunking: { enum: ["fixed-size", "sentence-aware", "semantic"]
},
      chunk_texto: { bsonType: "string", minLength: 1 },
      embedding: { bsonType: "array", minItems: 384, maxItems: 384 },
      modelo: { bsonType: "string" }
    }
  }},
  validationLevel: "strict",
```

```
  validationAction: "error"
});
```
[Reglas para las demás colecciones]

5. Configuración del entorno

Elemento	Valor
Opción	[A: Atlas M0 / B: Docker con MongoDB 7.0+]
Versión de MongoDB	[ ]
Región del cluster (si Atlas)	[ ]
Driver y versión	[pymongo / motor / mongodb para Node]
Scripts de inicialización	[ruta en el repositorio: init.py / init.js]
Gestión de credenciales	[.env fuera del repositorio]

6. Plan de experimento de chunking

Deben implementar al menos dos estrategias; se recomienda comparar las tres. Si cambian los parámetros sugeridos, justifiquen el cambio.

6.1 Estrategias y parámetros

Estrategia	¿Se implementará?	Parámetros sugeridos	Parámetros del grupo	Librería	Por qué podría funcionar en su dominio
Fixed-size	[Sí / No]	chunk_size = 256 tokens, overlap = 32	[ ]	[langchain text_splitter]	[ ]
Sentence-aware	[Sí / No]	máx. 5 oraciones, overlap = 1 oración	[ ]	[langchain / nltk]	[ ]
Semantic	[Sí / No]	umbral coseno entre 0.75 y 0.85	[ ]	[semantic-chunkers]	[ ]

Hipótesis: [qué estrategia esperan que funcione mejor con sus documentos y por qué.]